

Linked Stacks in C

Dynamic Representation & Expression Conversion

Unit I • Lecture 9

init • isEmpty • size • push • pop • peek • clear • display + infix $\rightarrow$ postfix / prefix



Why replace the array with linked nodes?

Array stack (Lecture 8)

Capacity fixed by `STACK_CAPACITY` at compile time. Push fails once the array is full, even if memory is free.

Linked stack (today)

One node per element, created with `malloc` at push and released with `free` at pop. The stack grows and shrinks at run time.

- Same interface, new representation — the ADT promise from Lecture 2.
- No `isFull( )`: the only limit is heap memory, reported by `malloc` returning `NULL`.
- Every operation still runs in $O(1)$, except `display` and `clear` which visit each node once.

Two structs describe the whole stack

```
#include <stdio.h>
#include <stdbool.h>
#include <stddef.h>
#include <stdlib.h>

typedef struct Node {
    int      value;
    struct Node *next;
} Node;

typedef struct {
    Node    *top;
    size_t   size;
} Stack;
```

- Node holds one value and the link to the node below it.
- struct Node *next is legal inside the struct because the tag names the type being defined.
- Stack keeps two facts: the top pointer and a running size counter.
- stdlib.h is new — it declares malloc and free.

The interface is unchanged from Lecture 8

```
void    stackInit(Stack *s);
bool    stackIsEmpty(const Stack *s);
size_t  stackSize(const Stack *s);

bool    stackPush(Stack *s, int value);
bool    stackPop(Stack *s, int *outValue);
bool    stackPeek(const Stack *s,
                  int *outValue);

void    stackClear(Stack *s);
void    stackDisplay(const Stack *s);
```

What changed?

Only one prototype is gone:
stackIsFull.

Every other signature is
identical, so Lecture 8's main()
still compiles against this stack.

stackInit, stackIsEmpty and stackSize

```
void stackInit(Stack *s) {  
    s->top = NULL;  
    s->size = 0;  
}
```

```
bool stackIsEmpty(const Stack *s) {  
    return s->top == NULL;  
}
```

```
size_t stackSize(const Stack *s) {  
    return s->size;  
}
```

- NULL is the empty sentinel — the linked answer to top = -1.
- size is maintained by push and pop, so reading it costs $O(1)$.
- const marks the two query functions as read-only, exactly as in Lecture 8.

stackPush links the new node in front

```
bool stackPush(Stack *s, int value) {
    Node *node = malloc(sizeof *node);
    if (node == NULL)
        return false;          /* heap full */

    node->value = value;
    node->next  = s->top; /* link first */
    s->top      = node;   /* then move */
    s->size    += 1;
    return true;
}
```

Order of the two pointer writes



Classic exam trap

Writing `s->top = node` before `node->next = s->top` loses the entire old chain — link first, then move top.

No overflow check — malloc's NULL result is the only capacity limit.

stackPop unlinks, reports, and frees

```
bool stackPop(Stack *s, int *outValue) {
    if (stackIsEmpty(s) || outValue == NULL)
        return false;      /* underflow */

    Node *node = s->top;    /* keep hold */
    *outValue = node->value;
    s->top = node->next;
    free(node);             /* release */
    s->size -= 1;
    return true;
}
```

- Keep a copy of the old top before moving the pointer — otherwise the node becomes unreachable and can never be freed.
- `free(node)` returns the memory; skipping it leaks one node per pop.
- The `outValue` `NULL` guard matches the Lecture 8 reference file.

stackPeek reads through the top pointer

```
bool stackPeek(const Stack *s,
               int *outValue) {
    if (stackIsEmpty(s) || outValue == NULL)
        return false;

    *outValue = s->top->value;
    return true;
}
```

- No node is created or freed — peek allocates nothing.
- top does not move; the stack is unchanged.
- const Stack *s lets the compiler enforce that promise.

s->top->value, read left to right

follow top to the first node, then take its value field — two -> hops, no loop.

stackClear and stackDisplay walk the chain

```
void stackClear(Stack *s) {
    Node *node = s->top;
    while (node != NULL) {
        Node *next = node->next;
        free(node);
        node = next;
    }
    s->top = NULL;
    s->size = 0;
}
```

```
void stackDisplay(const Stack *s) {
    if (stackIsEmpty(s)) {
        printf("[empty]\n");
        return;
    }
    printf("TOP -> ");
    for (const Node *n = s->top;
         n != NULL; n = n->next)
        printf("%d ", n->value);
    printf("\n");
}
```

Save next before free — a freed node must not be read again. Display uses its own walking pointer `n` and never moves `top`. Unlike the array version, clear must visit every node: $O(n)$.

Trace: three pushes and one pop

stackInit

NULL

top = NULL

push(10)



NULL

size 1

push(20)



NULL

size 2

push(30)



20

10

NULL

size 3

pop → 30



10

NULL

node freed

One program exercises every function

```
int main(void) {  
    Stack s;  
    int x;  
    stackInit(&s);  
  
    stackPush(&s, 10);  
    stackPush(&s, 20);  
    stackPush(&s, 30);  
    stackDisplay(&s);  
  
    if (stackPeek(&s, &x))  
        printf("Peek: %d\n", x);  
    if (stackPop(&s, &x))  
        printf("Pop:  %d\n", x);  
  
    printf("Size: %zu\n", stackSize(&s));  
    stackClear(&s);  
    stackDisplay(&s);  
    return 0;  
}
```

Program output

```
TOP -> 30 20 10  
Peek: 30  
Pop:   30  
Size: 2  
[empty]
```

Identical driver code to Lecture 8 — only the representation underneath changed.

Array stack vs linked stack

Aspect	Array (Lecture 8)	Linked (Lecture 9)
Capacity	Fixed: <code>STACK_CAPACITY</code>	Grows until heap is exhausted
Empty test	<code>top == -1</code>	<code>top == NULL</code>
Full test	<code>top == STACK_CAPACITY - 1</code>	<code>malloc</code> returns <code>NULL</code>
Memory per element	One int slot	int + next pointer + heap overhead
push / pop / peek	$O(1)$	$O(1)$
clear	$O(1)$ (reset top)	$O(n)$ (free every node)

First application: rewriting expressions

- Humans write infix: the operator sits between operands, so $A + B * C$ needs precedence rules and parentheses to be unambiguous.
- Machines prefer postfix (operator after operands) or prefix (operator before): no parentheses, no precedence table at run time.
- A stack of operators performs the conversion in one left-to-right scan.

INFIX $A + B * C$ **POSTFIX** $A B C * +$ **PREFIX** $+ A * B C$

Infix $\rightarrow$ postfix: the operator-stack rules

- 1 Operand** Append it straight to the output.
- 2 (** Push it; it waits for its matching **)**.
- 3 Operator** Pop to output while the stack top has higher precedence, or equal precedence and the operator is left-associative; then push it.
- 4)** Pop to output until **(** appears; discard the pair.
- 5 End of input** Pop everything that remains to the output.

Precedence: $^$ > $$ / > $+$ - $^$ is right-associative; the rest are left-associative.*

Worked example: $A + B * C - D$

Symbol	Action	Stack	Output
A	operand → output		A
+	stack empty → push	+	A
B	operand → output	+	A B
*	* higher than + → push	+ *	A B
C	operand → output	+ *	A B C
-	pop * then + ($\geq$ prec) → push -	-	A B C * +
D	operand → output	-	A B C * + D
end	pop the rest		A B C * + D -

Infix $\rightarrow$ prefix: reverse, convert, reverse

1

Reverse the infix string and swap every (with).

$$\begin{array}{l} A + B * C - D \\ \rightarrow D - C * B + A \end{array}$$

2

Run the postfix algorithm; on equal precedence, push instead of pop.

gives:
 $D C B * + A -$

3

Reverse the result — this is the prefix form.

$$- + A * B C D$$

Check: $- + A * B C D$ reads as $(A + (B \times C)) - D$ — the same expression, operators in front.

One interface, two representations, one application

- A linked stack keeps the Lecture 8 interface but swaps the array for malloc'd nodes: top == NULL is empty, malloc failure is the only "full".
- push links the new node before moving top; pop saves the node, moves top, then frees it — order matters in both.
- clear must free every node ($O(n)$); display walks with its own pointer and changes nothing.
- An operator stack converts infix to postfix in one scan; prefix is reverse $\rightarrow$ convert $\rightarrow$ reverse.

EXIT TICKET Convert $(A + B) * C - D / E$ to postfix and prefix, showing the stack at each step.